

Stack unwinding

Stack Unwinding is a process that occurs when an exception happens in a program.

What is the Call Stack?

- The call stack is like a stack of plates. When a function (or method, in Java) is called, it gets placed on top of the stack. When the function finishes running, it gets removed (or "popped") from the stack.
- Each function call keeps track of where it should return after finishing its job. This is known as the "return address."

How Does the Call Stack Work?

- **Imagine you have three methods: A, B, and C. If A calls B, and B calls C, the call stack will look like this:**

```
| C | <- Top of the stack (currently running)
| B |
| A |
```

What Happens During an Exception?

1. Throwing an Exception:

- If something goes wrong in one of these methods (let's say C throws an exception), Java looks for a way to handle this exception. This is where stack unwinding comes into play.

2. Unwinding the Stack:

- Java starts to "unwind" the stack. This means it goes back down the stack, looking for a method that has a way to handle the exception (using a try-catch block).
- The stack is popped one method at a time. If method C can't handle the exception, Java looks to method B, and then to method A.

3. Finding a Handler:

- If it finds a try-catch block that matches the exception, it will execute that block to handle the error. If it doesn't find any

handler after going through all the methods, it will terminate the program and print the exception message.

Example

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            methodA();  
        } catch (Exception e) {  
            System.out.println("Caught an exception: " +  
e.getMessage());  
        }  
    }  
  
    public static void methodA() {  
        methodB();  
    }  
  
    public static void methodB() {  
        methodC();  
    }  
  
    public static void methodC() {  
        throw new RuntimeException("Something went wrong!");  
    }  
}
```

In this example:

- main calls methodA.
- methodA calls methodB.
- methodB calls methodC, which throws a RuntimeException.

The exception is thrown in methodC. Since methodC doesn't handle it, Java unwinds the stack:

1. methodC is removed.
2. methodB is removed.
3. methodA is removed.

Since methodA can't handle it either, it goes back to main, which has a try-catch block to catch the exception and print the message.

Summary

- **Stack unwinding** is the process of going back through the call stack to find a place to handle an exception.
- If no handler is found, the program will terminate.
- It's a way of ensuring that the program can gracefully handle errors rather than crashing unexpectedly.